

CHAPTER 2

线性表

孙焕良

sunhl@sjzu.edu.cn

2.1 线性表ADT

对象: $(A_1, A_2, \dots, A_N)$

操作:

☞ 返回长度, N .

☞ 输出所有元素.

☞ 清空一个线性表list.

☀☞ **Finding** 查找第 k -th, $1 \leq k \leq N$.

☀☞ **Inserting** 在第 k -th个元素后插入一个元素, $1 \leq k \leq N$.

☀☞ **Deleting** 删除一个元素.

☞ **Finding next** 找下一个元素.

☞ **Finding previous** 找前一个元素.

2.2 线性表的顺序表示-顺序表

- 结点按逻辑顺序依次存放在一组地址连续的存储单元中;
- 第*i*个数据元素 a_i 的存储位置为 $LOC(a_i)=LOC(a_1)+(i-1) \times l$
- 第*i*+1个数据元素的存储位置 $LOC(a_{i+1})$ 和第*i*个数据元素的存储位置 $LOC(a_i)$ 满足 $LOC(a_{i+1})=LOC(a_i)+l$

用结构进行表示和实现:

```
#define LIST_INIT_SIZE 100 //线性表初始空间, 单位空间的倍数
```

```
#define LISTINCREMENT 10 //分配空间增量
```

```
typedef struct Sqlist{
```

```
    elemtype *elem; //数组指针 存储空间起始地址
```

```
    int length;    //当前长度
```

```
    int listsize;  //当前分配空间
```

```
} Sqlist;
```

例如: Sqlist L;

顺序表上的基本操作

1、初始化：构造一个空表。

```
Status InitList_Sq(Sqlist &L){  
    L.elem=(ElemType*)malloc(sizeof(ElemType)*LIST_INIT_SIZE);  
    if (!L.elem )  
        exit(OVERFLOW);  
    L.length=0;  
    L.listsize=LIST_INIT_SIZE;  
    return OK;  
}
```

顺序表上的基本操作

2、插入：在表的第i ($1 \leq i \leq n+1$)个位置插入一个新结点e

```
Status ListInsert_Sq(Sqlist &L, ElemType e, int i) {
    elemtype *p,*q,*newbase;
    if(i<1 || i>L.length+1){
        printf("position error");
        return ERROR; }
    if(L.length>=L.listsize){
        newbase=(ElemType*)realloc(L.elem,
            (L.listsize+LISTINCREMENT)*sizeof(ElemType));
        if (!newbase) exit ( OVERFLOW );
        L.elem=newbase;
        L.listsize+=LISTINCREMENT;
    }
    q=&(L.elem[i-1]); //q为插入位置
    for(p=&(L.elem[L.length-1]); p>=q; --p)
        *(p+1)=*p;
    *q=e;
    ++L.length;
    return OK;
}
```

- 问题规模是表的长度 n ，移动结点的次数 $n-i+1$ ；
- 当 $i=n$ 时，时间复杂度 $O(1)$ ；
- 当 $i=1$ 时，时间复杂度为 $O(n)$ ；
- 设 p_i 是在第 i 个元素之前插入一个元素的概率，在长度为 n 的线性表中插入一个元素时，所需移动的元素次数 复杂度为 $O(n)$

$$\sum_{i=1}^{n+1} p_i (n-i+1) = \sum_{i=1}^{n+1} \frac{1}{n+1} (n-i+1) = \frac{1}{n+1} \sum_{i=1}^{n+1} (n-i+1) = \frac{1}{n+1} \frac{n(n+1)}{2} = \frac{n}{2}$$

3、删除 第 $i(1 \leq i \leq n)$ 结点删除 平均次数 复杂度为 $O(n)$

$$\sum_{i=1}^n p_i (n-i) = \sum_{i=1}^n \frac{1}{n} (n-i) = \frac{1}{n} \sum_{i=1}^n (n-i) = \frac{1}{n} \frac{n(n-1)}{2} = \frac{n-1}{2}$$

4、按值查找 平均次数 复杂度为 $O(n)$

$$\sum_{i=1}^n p_i \times i = \sum_{i=1}^n \frac{1}{n} \times i = \frac{1}{n} \frac{n(n+1)}{2} = \frac{n+1}{2}$$

顺序表-数组实现

$\text{array}[i] = \text{item}_i$

顺序映像

地址	内容
.....	
$\text{array}+i$	item_i
$\text{array}+i+1$	item_{i+1}
.....	



MaxSize，估计最大长度



Find_Kth， $O(1)$ 随机存取



插入与删除要大量移动



Experiment 2-0 力扣27. 移除元素

描述：给你一个数组 `nums` 和一个值 `val`，需要原地移除所有数值等于 `val` 的元素，并返回移除后数组的新长度。不要使用额外的数组空间，仅使用 $O(1)$ 额外空间并原地 修改输入数组。

输入：nums = [0,1,2,2,3,0,4,2], val = 2

输出：5, nums = [0,1,4,0,3]

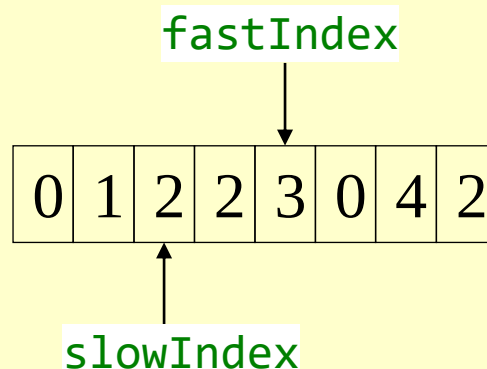
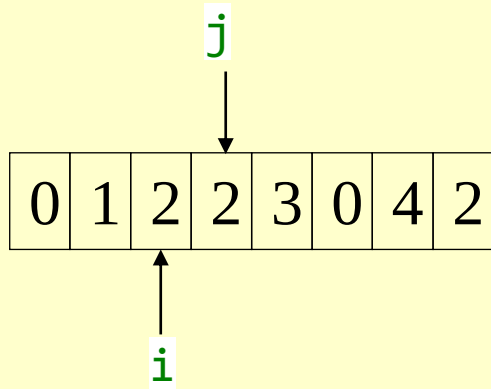
解释：函数应该返回新的长度 5，并且 `nums` 中的前五个元素为 0, 1, 3, 0, 4。五个元素可为任意顺序。不需要考虑数组中超出新长度后的元素。

```
int removeElement(int* nums, int numsSize, int val)
```

方法1：暴力方法，利用双重循环实现

过程先画出来，再写程序

方法2：双指针，利用快慢指针单循环实现



2.3 链式结构

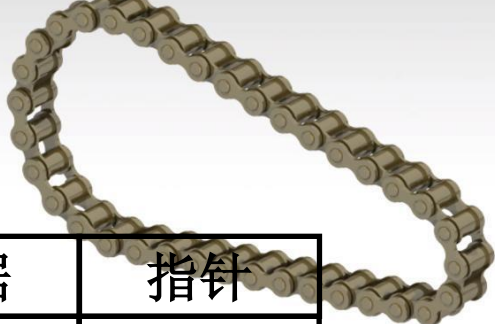
一、单链表

地址	数据	指针
0010	a3	1011
0011	a2	0010
0110	a1	0011
1011	a4	NULL

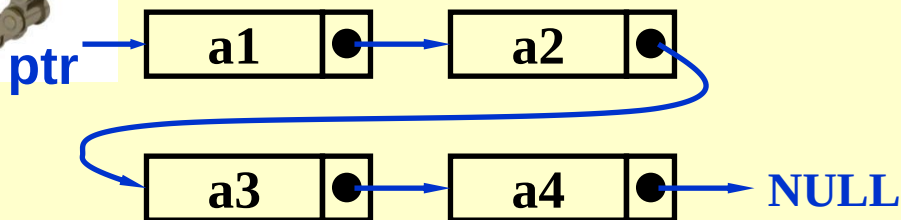
头指针 ptr = 0110

类型声明:

```
typedef struct LNode {  
    ElemType data;  
    struct LNode *next;  
} LNode, *LinkList;
```

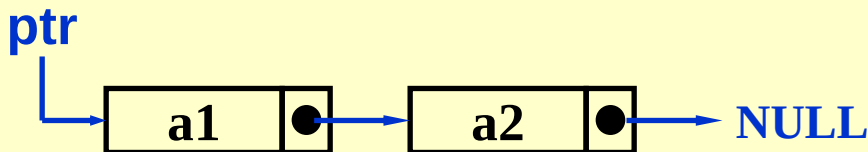


ADT

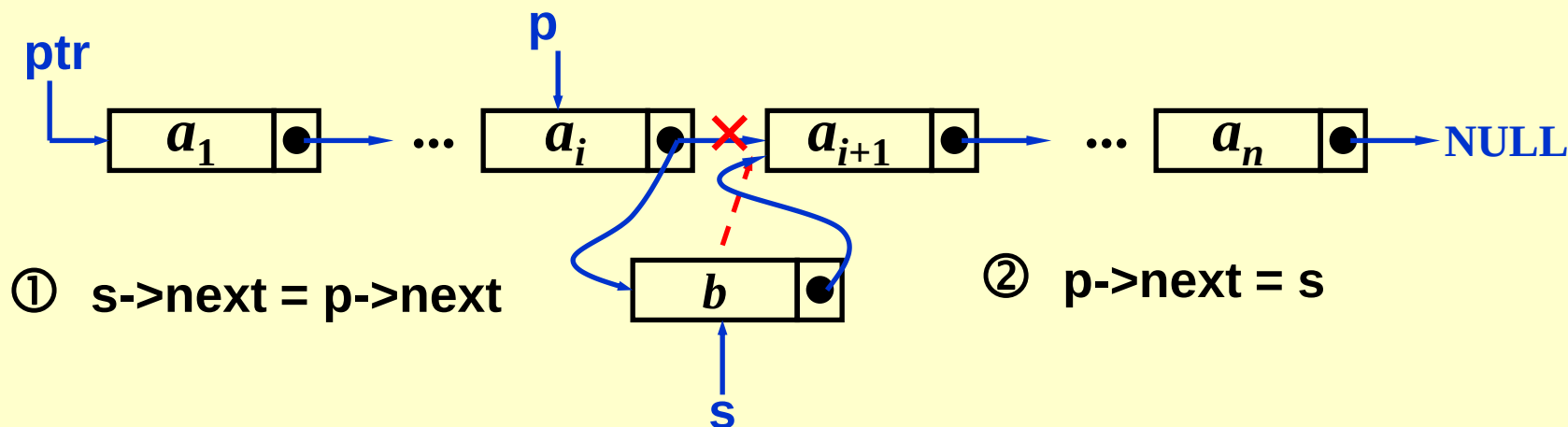


连接'a1' and 'a2':

```
LinkList N1, N2 ;  
N1 = (LinkList)malloc(sizeof(struct LNode));  
N2 = (LinkList)malloc(sizeof(struct LNode));  
N1->data = 'a1' ;  
N2->data = 'a2' ;  
N1->next = N2 ;  
N2->next = NULL ;  
ptr = N1 ;
```



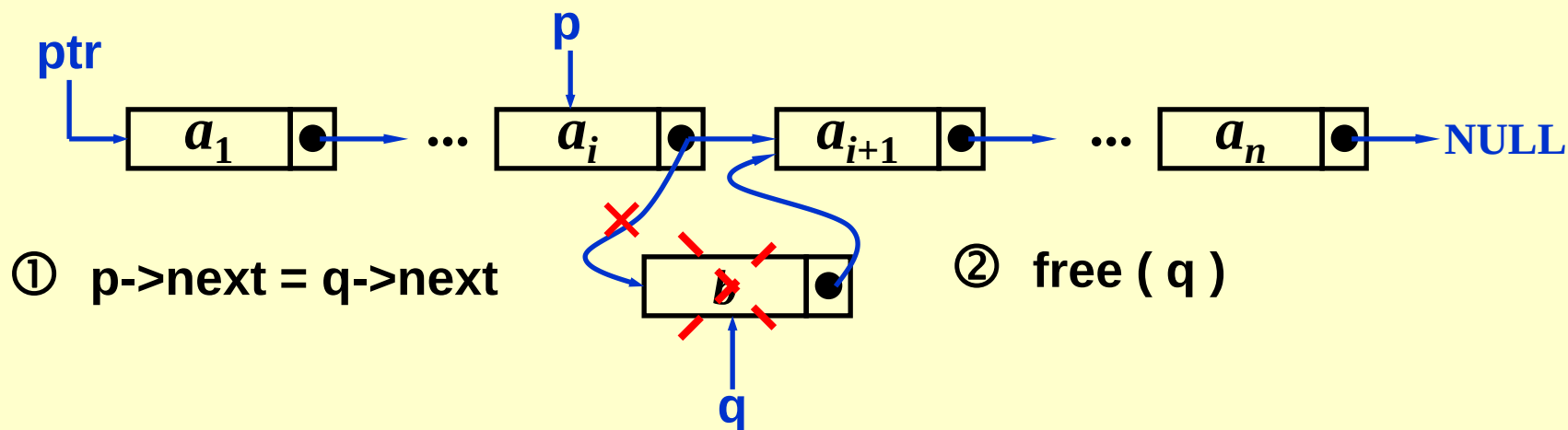
单链表基本操作1: 插入 找到i位置 $O(n)$ 插入 $O(1)$



Question: 次序交换会怎么样?

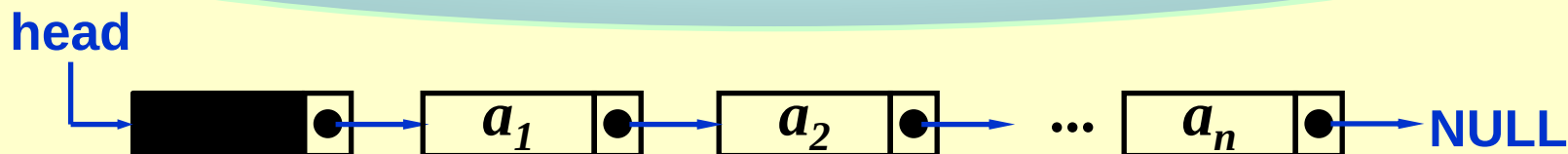
Question: 如何插入元素作为第一个元素?

单链表基本操作2:删除 找到 $i-1$ 位置 $O(n)$ 插入 $O(1)$

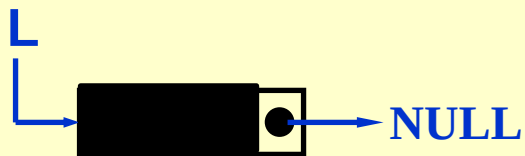


Question: 如何删除第一个元素?

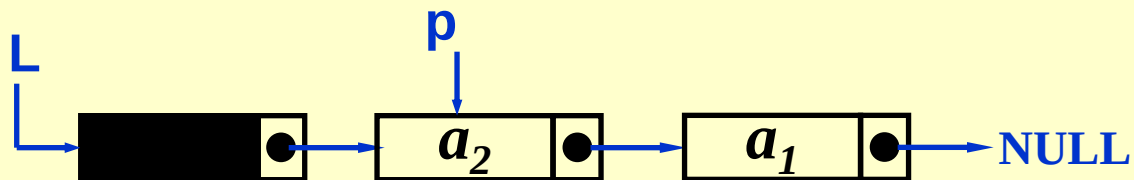
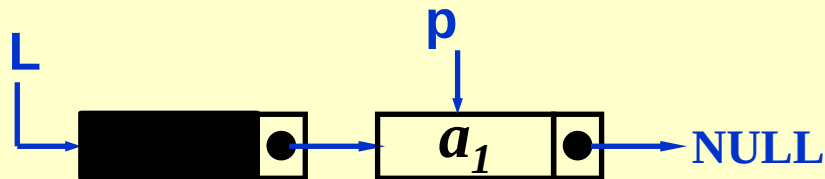
Answer: 加一个不存储数据的头结点。



单链表基本操作3: 创建 (头插法) $O(n)$



```
p = (LinkList)malloc(sizeof(LNode));  
scanf("%d", &p->data);  
p->next = L->next;  
L->next = p;
```



单链表基本操作4: 创建 (尾插法) $O(n)$

单链表基本操作5: 按值查找表结点 $O(n)$

单链表基本操作6: 求表长 $O(n)$

单链表与顺序表比较

	顺序表	链表
空间分配方式	一次性分配	多次分配
空间存储密度	1	<1
存取方式	随机，顺序	顺序
插入/删除移动元素	平均移动近一半	不移动元素
各有优缺点		

Experiment 2-1 Linked list implementation

函数:

1. Create a list; (头插法创建)
2. Create a list; (尾插法创建)
3. Display a list; (显示)
4. Insert an element; (插入)
5. Delete an element; (删除特定值结点)
6. Get an element; (返回给定位置值或定位给定值)
7. Clear a list;
0. Exit;

Experiment 2-1 Linked list implementation (扩展)

函数:

8. 删除倒数第n个元素; 力扣19 (双指针, 等距向前移动)
9. Merge two lists; (合并教材p39, 力扣21)
(两个有单链序表按序合并)
10. Reverse a list; (反转) 力扣206 (双指针, 不用新建)
11. 判断单链表中是否有环; 力扣142 (快慢指针, 相遇则有环)

删除链表的倒数第 n 个结点：给定一个链表，删除链表的倒数第 n 个结点，并且返回链表的头结点。

输入： head = [1,2,3,4,5,6,7], $n = 2$ **输出：** [1,2,3,4,5,7]

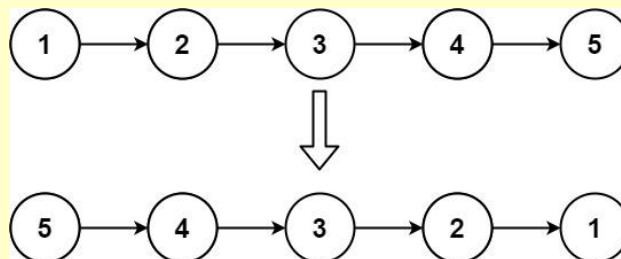
合并两个有序链表：将两个升序链表合并为一个新的升序链表并返回。新链表是通过拼接给定的两个链表的所有节点组成的。

输入： l1 = [1,2,6], l2 = [3,5,8,9,10] **输出：** [1,2,3,5,6, 8,9,10]

反转链表：给你单链表的头节点 **head**，请你反转链表，并返回反转后的链表。

输入： **head = [1,2,3,4,5]**

输出： **[5,4,3,2,1]**



判断单链表中是否有环： 给定一个链表的头节点 head，返回链表开始入环的第一个节点。如果链表无环，则返回 null。

输入： head = [0,1,2,3,4,5,6,**3**], pos = 3

输出： 返回索引为 3 的链表节点

slow 1步

fast 2步

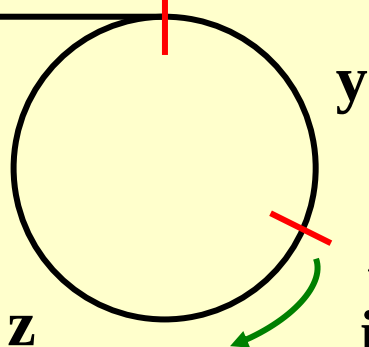
index2



x

入口

slow fast



相遇条件： slow==fast

入口位置： index1==index2

slow: $x+y$

1次1节点

fast: $x+y+n(y+z)$

1次2节点

$$(x+y)*2 = x+y+n(y+z)$$

$$x+y = n(y+z)$$

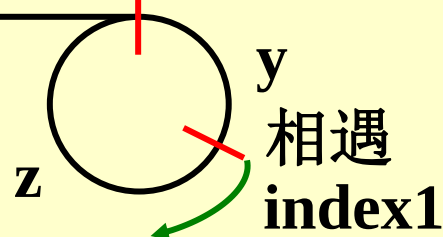
$$x = (n-1)(y+z) + z$$

index2



x

入口



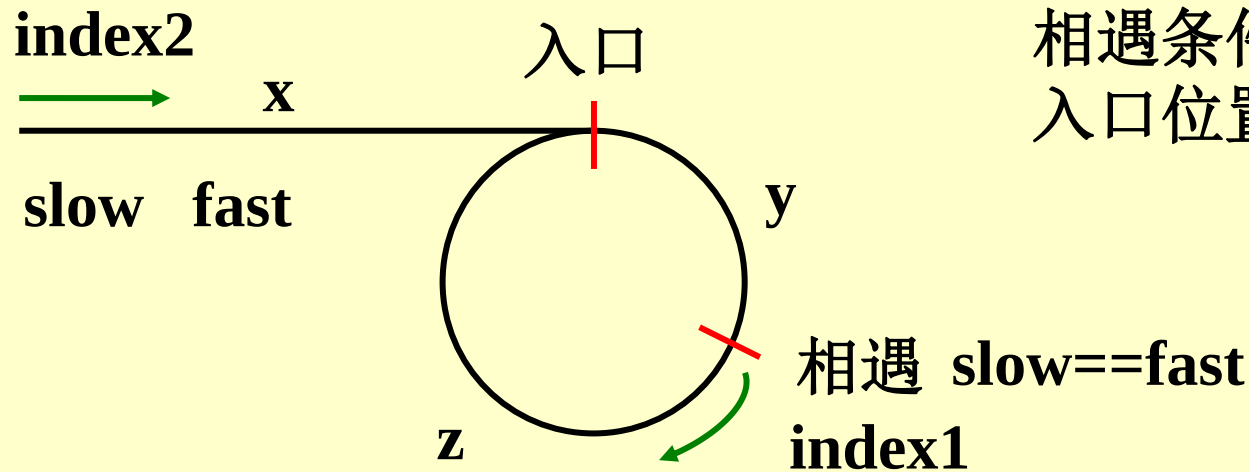
$n>1$

判断单链表中是否有环： 给定一个链表的头节点 head，返回链表开始入环的第一个节点。如果链表无环，则返回 null。

输入： head = [0,1,2,3,4,5,6,**3**], pos = 3

输出： 返回索引为 3 的链表节点

slow 1步
fast 2步

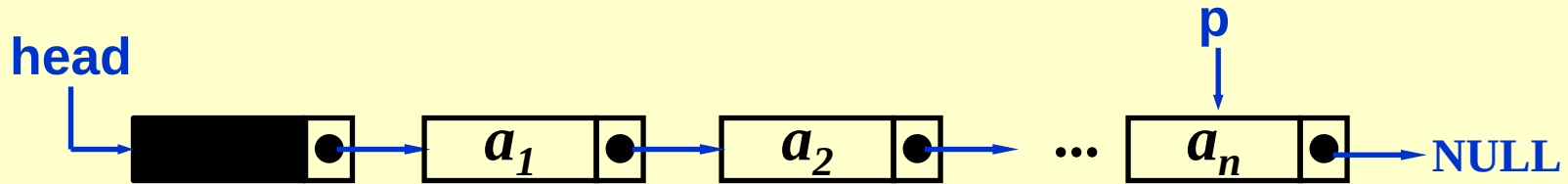


相遇条件： slow==fast
入口位置： index1==index2

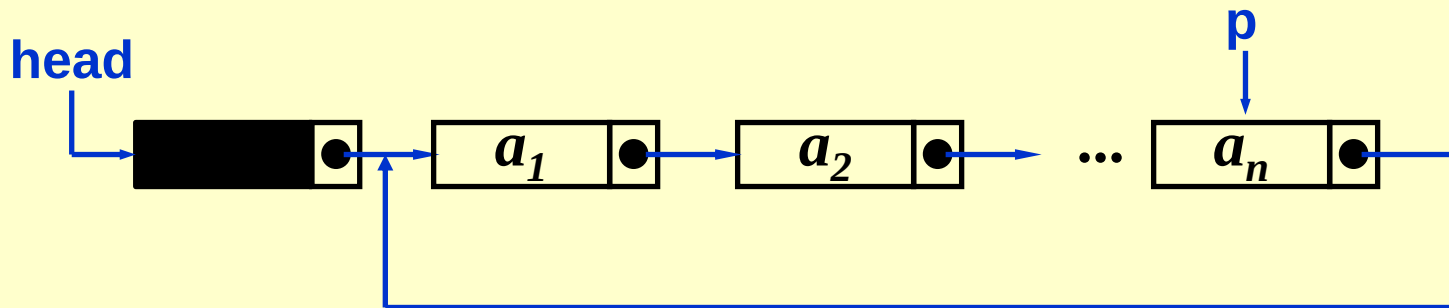
//创建一个有环的单链表，用尾插法。

LinkedList creatListCycle(LinkedList L, int n, int m) //m为环所在的位置

二、单向循环链表Single Linked Circular Lists

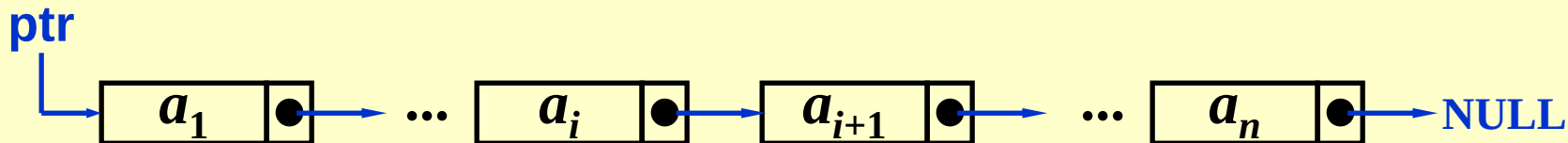


□ 当p指向尾部后，如何找第一个结点？



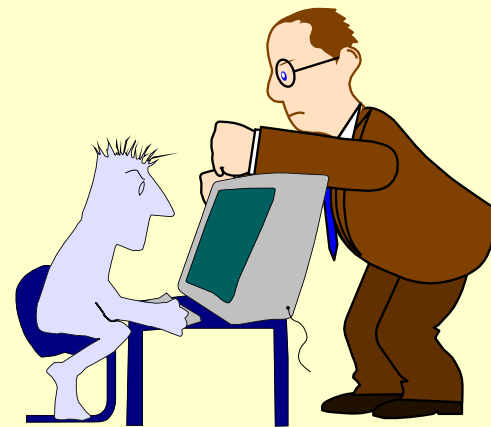
$p \rightarrow \text{next} = \text{head} \rightarrow \text{next}$

三、双向循环链表Doubly Linked Circular Lists

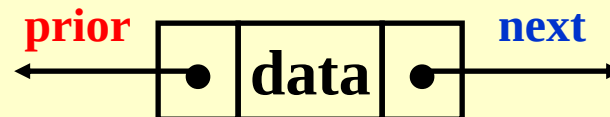


为什么需要双向循环链表?

□ 假定 $a_1 \rightarrow a_2 \rightarrow a_3 \rightarrow \dots \rightarrow a_m$ ，如何找一个结点的前一个结点?

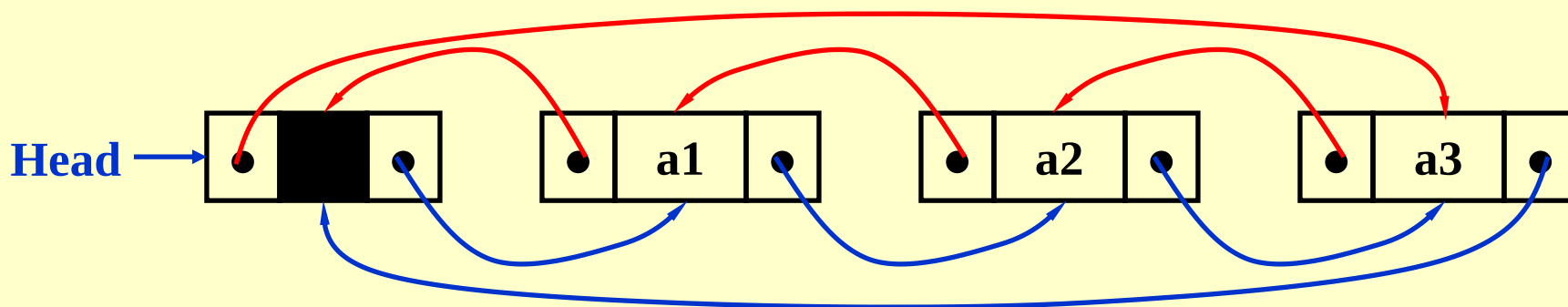


```
typedef struct DListNode {
    struct DListNode *prior;
    ElemType data;
    struct DListNode *next;
} DListNode, * DLinkList;
```

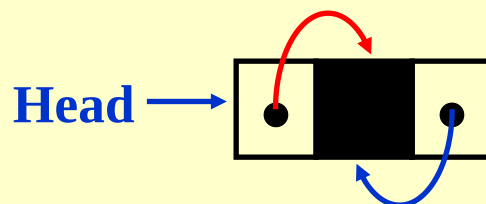


$p = (p \rightarrow \text{prior}) \rightarrow \text{next}$
 $= (p \rightarrow \text{next}) \rightarrow \text{prior}$

带头结点的双向循环链表:

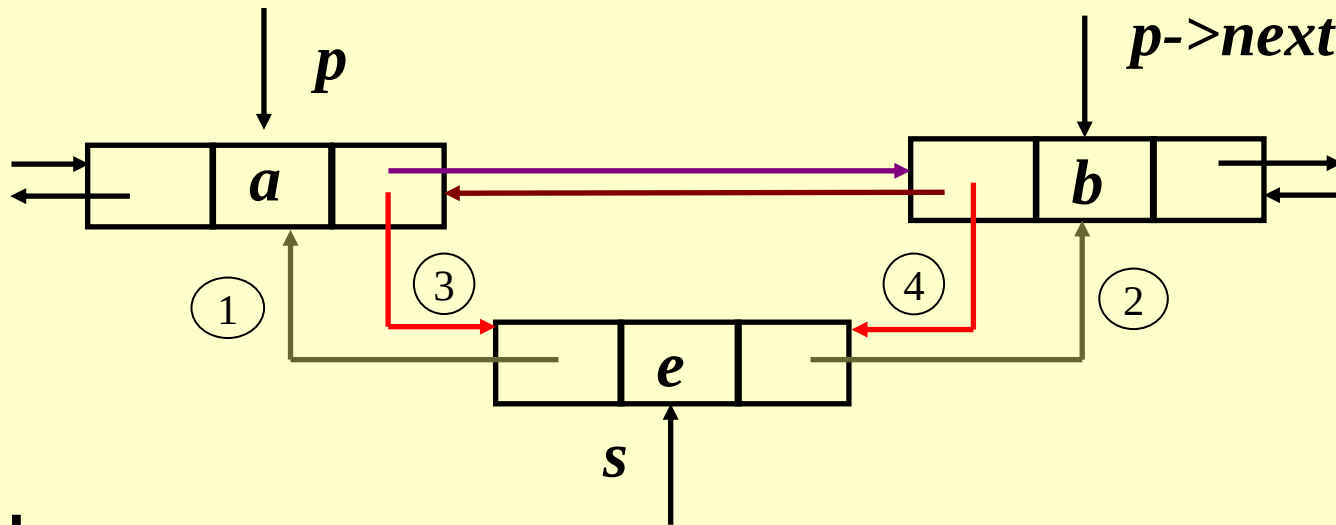


空表:



空表条件?

Insertion插入：在**p**后插入一个结点，值为**e**.



Algorithm:

```
s = (DLinkedList)malloc(sizeof(DListNode));
```

```
s->data = e;
```

```
s->prior = p;
```

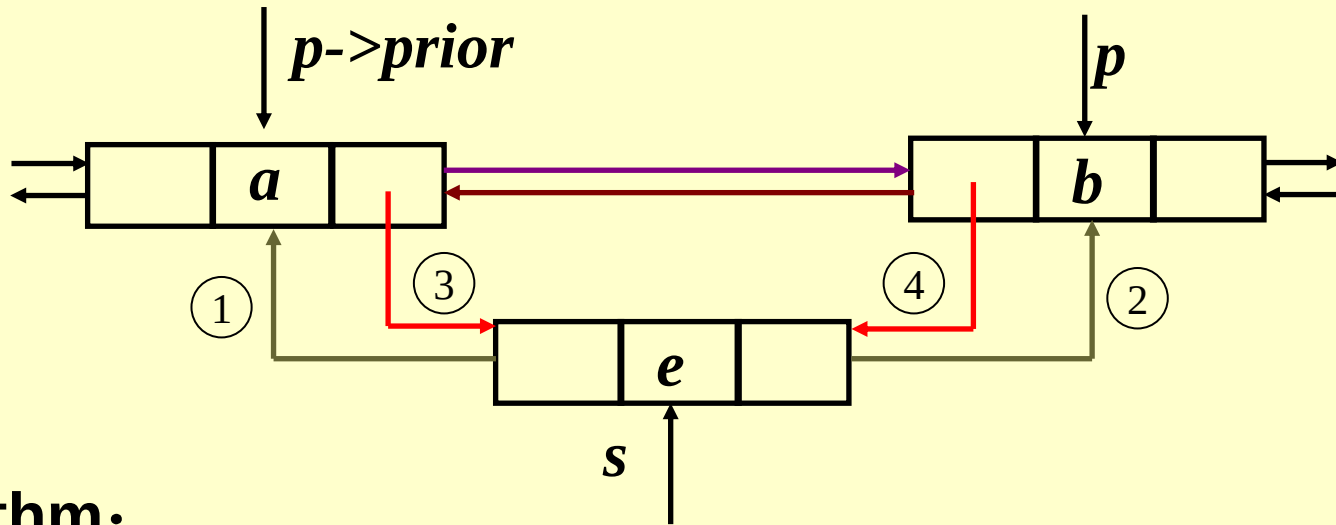
```
s->next = p->next;
```

```
p->next = s;
```

```
s->next->prior = s;
```

Q: 能否改变操作次序?

Insertion插入：在**p前**插入一个结点，值为e.



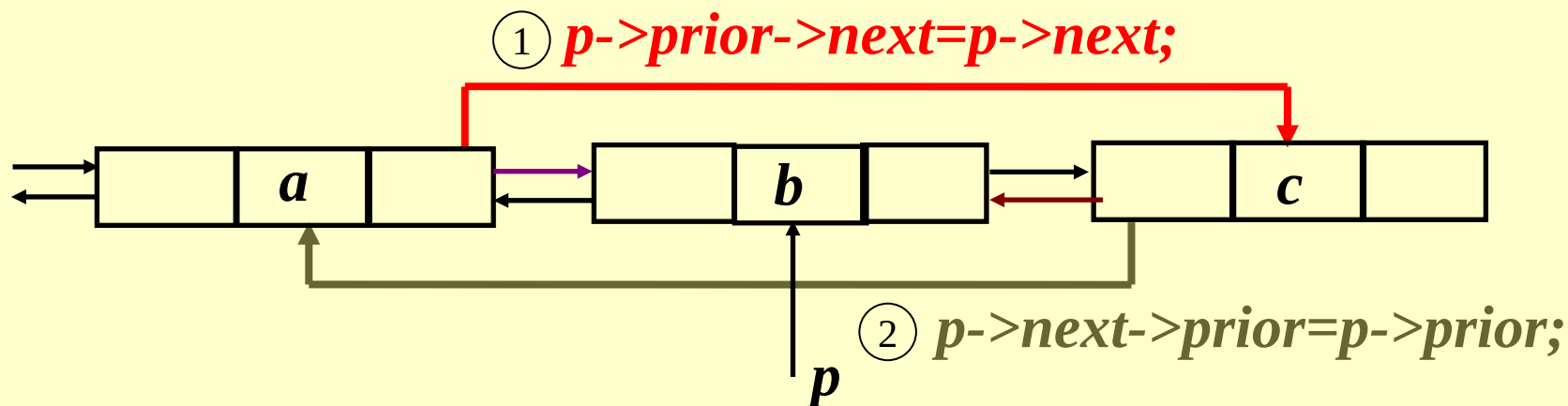
Algorithm:

```
s = (DLinkedList)malloc(sizeof(DListNode));
```

```
s->data = e;
```

```
_____;  
_____;  
_____;  
_____;
```

Deletion删除: 删除p指向的结点



Algorithm:

```
p->prior->next=p->next;  
p->next->prior=p->prior;  
free(p);
```

能否改变操作次序?

四、静态链表

定义:

```
#define MAXSIZE 1000 /*足够大的数*/  
typedef struct{  
    ElemType data;  
    int next;  
} component,  
SlinkNode[MAXSIZE];
```

这种链表的结点中也有数据域data和指针域next，与前面所讲的链表中的指针不同，该指针是结点的相对地址(数组的下标)，称之为**静态指针**，这种链表称之为**静态链表**。

空指针用0表示,表明表尾。

数组的第零分量看作**头结点**。

	<i>data</i>	<i>next</i>
0		1
1	zhao	3
2	li	5
3	qian	4
4	sun	2
5	zhou	0

与其他线性表的区别

- 与链表的主要区别

动态链表	静态链表
结点是系统分配	用户事先定义
结点地址是与该类型同类型的指针表示	数组下标
存储地址不一定连续	地址连续

- 与线性表的主要区别：物理地址连续，但逻辑结构不一定连续。

Experiment 2-2 Double Linked list implementation

函数： 在单链表基础上进行修改

1. Create a list; (头插法)
2. Create a list by inserting from tail;(尾插法)
3. Display a list;
4. Display a list from tail;
5. Insert an element;
6. Delete an element;
7. Get an element;
8. Get neighbors of an element(前后)
9. Destroy a list;
0. Exit;\n");

2.4 典型应用

*应用一 多项式The Polynomial ADT ()

Objects : $P(x) = a_1 x^{e_1} + \dots + a_n x^{e_n}$; a set of ordered pairs of $\langle e_i, a_i \rangle$ where a_i is the **coefficient** 系数 and e_i is the **exponent** 指数. e_i are nonnegative integers.

Operations:

- ☞ 次 **Finding degree**, $\max \{ e_i \}$, of a polynomial.
- ☞ 加法 **Addition** of two polynomials.
- ☞ 减法 **Subtraction** between two polynomials.
- ☞ 乘法 **Multiplication** of two polynomials.
- ☞ 除法 **Differentiation** of a polynomial.

【表示1】

```
typedef struct {  
    int    CoeffArray [ MaxDegree + 1 ] ;  
    int    HighPower;  
} *Polynomial ;
```

□ It's easy to implement most of the operations, such as Add and Multiplication.

□ But try to apply MultPolynomial on

$$P_1(x) = 10x^{1000} + 5x^{14} + 1 \text{ and}$$

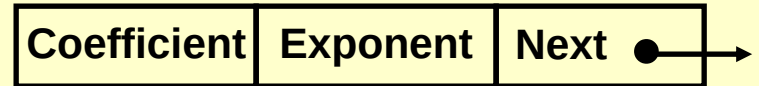
$$P_2(x) = 3x^{1990} - 2x^{1492} + 11x + 5$$

【表示 2】

Given: $A(x) = a_{m-1}x^{e_{m-1}} + \cdots + a_0x^{e_0}$

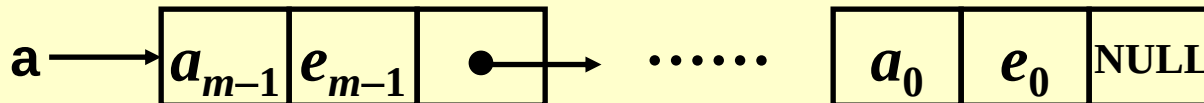
where $e_{m-1} > e_{m-2} > \cdots > e_0 \geq 0$ and $a_i \neq 0$ for $i = 0, 1, \dots, m-1$.

We represent each term as a node



Declaration:

```
typedef struct poly_node *poly_ptr;  
struct poly_node {  
    int    Coefficient ; /* assume coefficients are integers */  
    int    Exponent;  
    poly_ptr  Next ;  
};  
typedef poly_ptr  a ; /* nodes sorted by exponent */
```



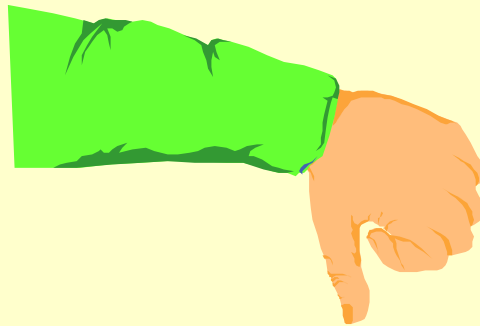
*应用二 多重链表Multilists

【Example】 Suppose that we have 40,000 students and 2,500 courses. **Print** the students' name list for each course, and **print** the registered classes' list for each student.

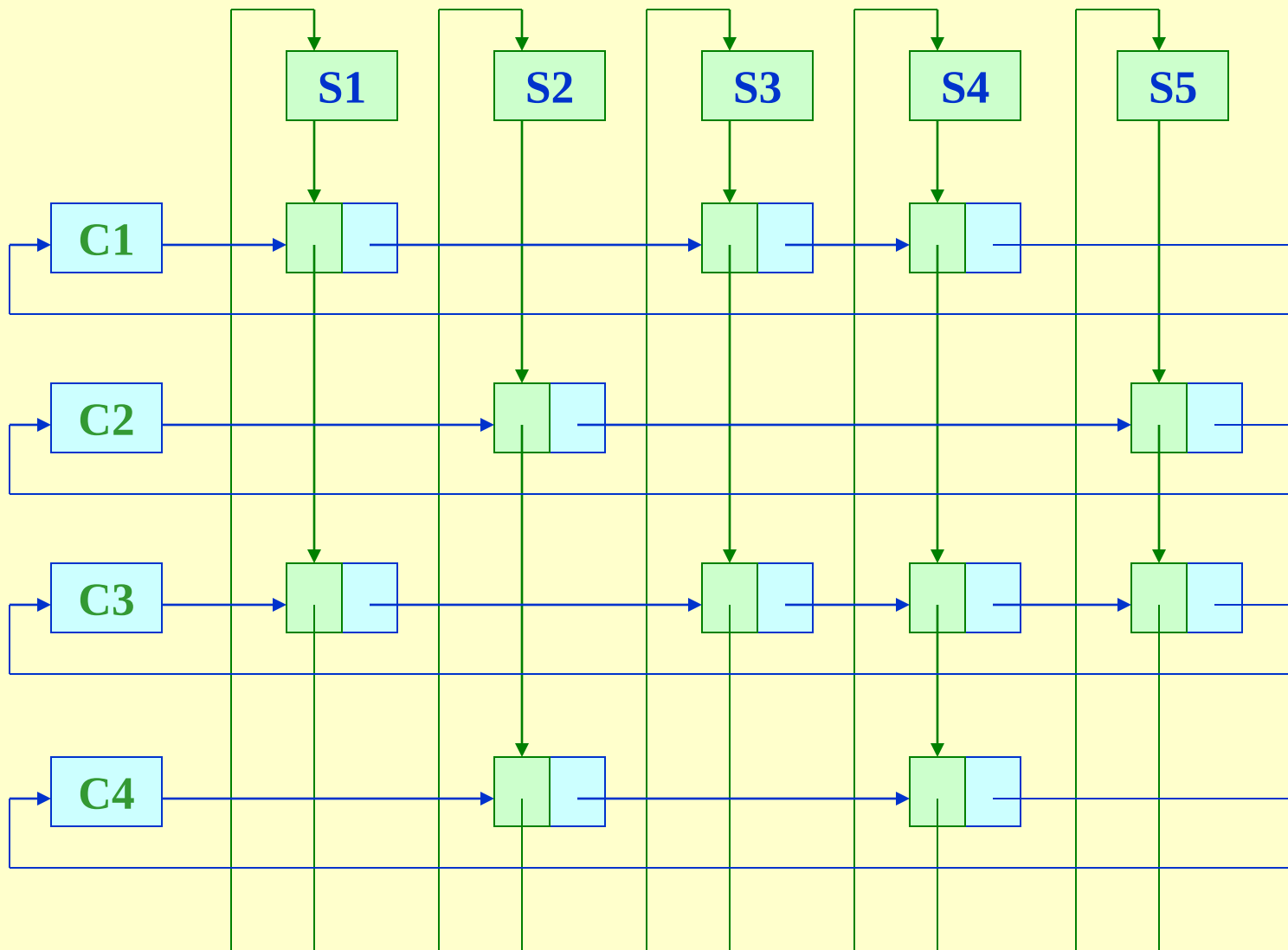
【Representation 1】

```
int Array[40000][2500];
```

$$\text{Array}[i][j] = \begin{cases} 1 & \text{if student } i \text{ is registered for course } j \\ 0 & \text{otherwise} \end{cases}$$



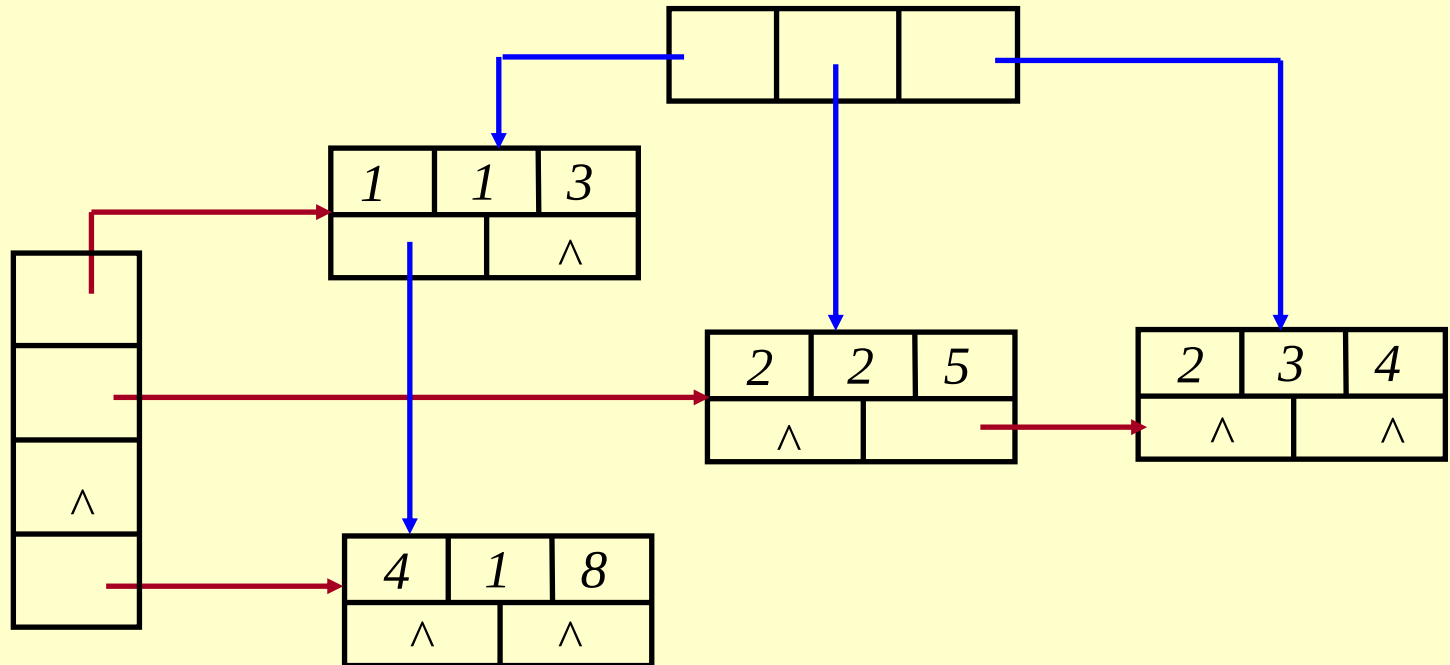
【表示 2】



$$A = \begin{bmatrix} 3 & 0 & 0 \\ 0 & 5 & 4 \\ 0 & 0 & 0 \\ 8 & 0 & 0 \end{bmatrix}_{4 \times 3}$$

Tag	row	column	value
	columnPtr		rowPtr

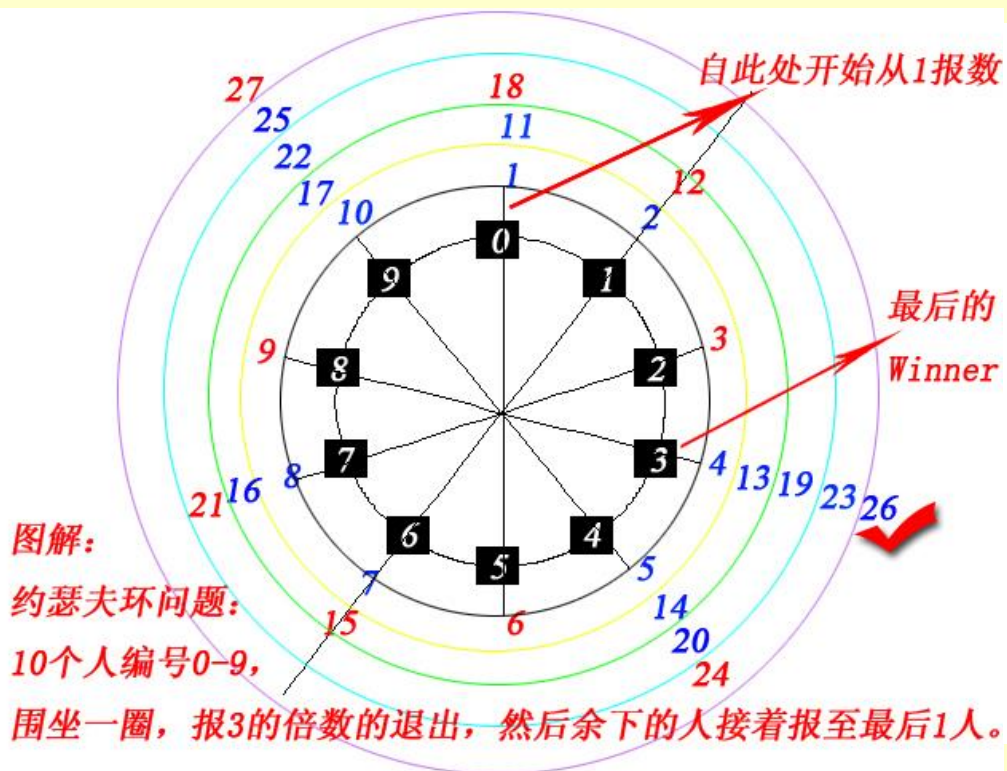
2	2	5
^		



Tag	next	
	columnPtr	rowPtr

✳ 应用三 约瑟夫问题Josephus Problem

Experiment 2-3 Josephus Problem (or Josephus permutation)



作业：

1 练习：2线性表

2 默写单链表与双向循环链表插入、删除算法；

3 名词：时间复杂度，空间复杂度，数据逻辑结构，数据物理结构，数据元素，数据项，单链表，单向循环链表，双向循环链表。

练习2-1：选择题

- 1、设顺序线性表中有 n 个数据元素，则删除表中第 i 个元素需要移动（ ）个元素。
(A) $n-i$ (B) $n+1-i$ (C) $n-1-i$ (D) i
- 2、带头结点的双向循环链表 L 为空的条件是（ ）。
A、 $L \rightarrow \text{prior} == L \ \&\& \ L \rightarrow \text{next} == \text{NULL}$ ； B、 $L \rightarrow \text{prior} == \text{NULL} \ \&\& \ L \rightarrow \text{next} == \text{NULL}$ ；
C、 $L \rightarrow \text{prior} == \text{NULL} \ \&\& \ L \rightarrow \text{next} == L$ ； D、 $L \rightarrow \text{prior} == L \ \&\& \ L \rightarrow \text{next} == L$ ；
- 3、带头结点的双向循环链表 L 为空时,以下判断为假的是（ ）。
A、 $L \rightarrow \text{prior} == \text{NULL} \parallel L \rightarrow \text{next} == L$ ； B、 $L \rightarrow \text{prior} == L \ \&\& \ L \rightarrow \text{next} == L$ ；
C、 $L \rightarrow \text{prior} == \text{NULL} \ \&\& \ L \rightarrow \text{next} == L$ ； D、 $L \rightarrow \text{prior} == L \parallel L \rightarrow \text{next} == L$ ；
- 4、[2016统考真题]带头结点的双向循环链表 L ，现要删除指针 p 所指的结点，正确的语句序列是（ ）。
A、 $p \rightarrow \text{next} \rightarrow \text{prior} = p \rightarrow \text{prior}$ ； $p \rightarrow \text{next} \rightarrow \text{prior} = p \rightarrow \text{prior}$ ； $\text{free}(p)$ ；
B、 $p \rightarrow \text{next} \rightarrow \text{prior} = p \rightarrow \text{next}$ ； $p \rightarrow \text{prior} \rightarrow \text{next} = p \rightarrow \text{next}$ ； $\text{free}(p)$ ；
C、 $p \rightarrow \text{next} \rightarrow \text{prior} = p \rightarrow \text{next}$ ； $p \rightarrow \text{prior} \rightarrow \text{next} = p \rightarrow \text{prior}$ ； $\text{free}(p)$ ；
D、 $p \rightarrow \text{next} \rightarrow \text{prior} = p \rightarrow \text{prior}$ ； $p \rightarrow \text{prior} \rightarrow \text{next} = p \rightarrow \text{next}$ ； $\text{free}(p)$ ；
- 5 某线性表用带头结点的循环单链表存储，头指针为 head ，当 $\text{head} \rightarrow \text{next} \rightarrow \text{next} == \text{head}$ 成立时，线性表长度为（ ）
A、0； B、1； C、2； D、0或1；
- 6、带头结点的双向循环链表 L 为空时,以下判断为假的是（ ）。
A、 $L \rightarrow \text{prior} == \text{NULL} \parallel L \rightarrow \text{next} == L$ ； B、 $L \rightarrow \text{prior} == L \ \&\& \ L \rightarrow \text{next} == L$ ；
C、 $L \rightarrow \text{prior} == \text{NULL} \ \&\& \ L \rightarrow \text{next} == L$ ； D、 $L \rightarrow \text{prior} == L \parallel L \rightarrow \text{next} == L$ ；

练习2-1：算法设计题

1. [2009统考真题]已知一个带有表头结点的单链表，假设该链表只给出了头指针head.在不改变链表的前提下，请设计一个尽可能高效的算法，查找链表中倒数第k个位置上的结点。若查找成功，算法输出该结点的数据域的值，并返回1；否则，只返回0.要求：

描述算法的基本设计思想；描述算法的详细实现步骤；采用程序设计语言描述算法，关键之处请给出简要注释。

2 设计一个算法找到单链表的中间结点，假设单链表有奇数个结点，要求一次遍历单链表。

3 设计一个算法完成以下功能，判断一个链表是否有环，如果有找出环的入口点并返回，否则返回NULL。